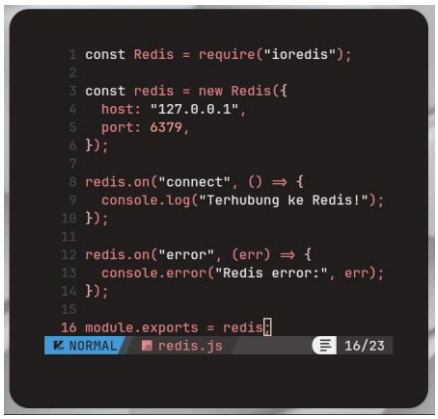
PRAKTIKUM
SISTEM BASIS DATA

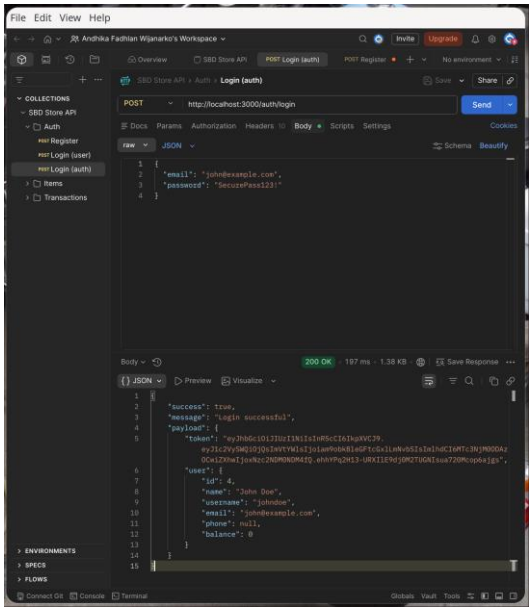
Nama	**Andhika Fadhlan Wijanarko**	**No. Modul**	**8**
NPM	**2306268164**	**Tipe**	**CS**

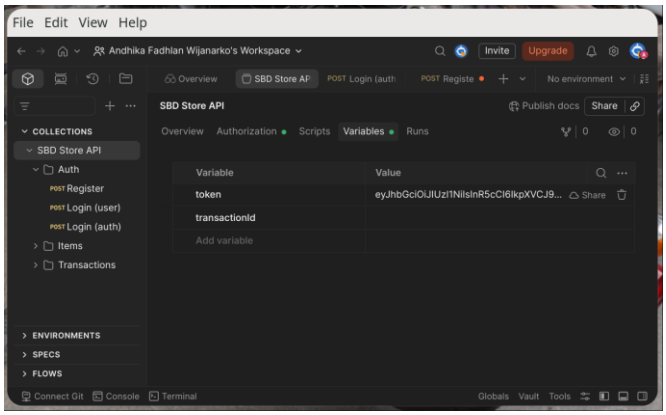
1. redis.js & /auth/login POST
   Screenshot:



```
const Redis = require("ioredis");

const redis = new Redis({
  host: "127.0.0.1",
  port: 6379,
});

redis.on("connect", () => {
  console.log("Terhubung ke Redis!");
});

redis.on("error", (err) => {
  console.error("Redis error:", err);
});

module.exports = redis;
```

NORMAL redis.js 16/23




File Edit View Help
Andhika Fadhlan Wijanarko's Workspace
Invite
Upgrade
Overview
SBD Store AP
POST Login (auth
POST Registe
No environment
SBD Store API
Publish docs
Share
COLLECTIONS
SBD Store API
Auth
POST Register
POST Login (user)
POST Login (auth)
Items
Transactions
Overview
Authorization
Scripts
Variables
Runs
Variable
Value
token
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Share
transactionId
Add variable
ENVIRONMENTS
SPECS
FLOWS
Connect Git
Console
Terminal
Globals
Vault
Tools

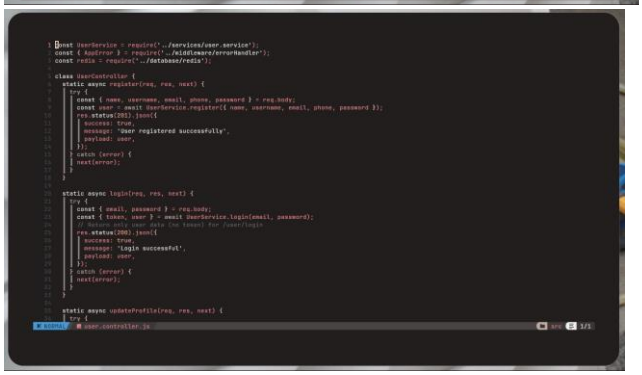
2. Hasil percobaan pertama dan kedua:

	{
	"success": true,
	"message": "User retrieved from database",
	"source": "database",
	"payload": {
	"id": 4,
	"name": "John Doe",
	"username": "johndoe",
	"email": "john@example.com",
	"phone": null,
	"balance": 0,
	"created_at": "2026-04-16T13:55:43.135Z"
	}
	}
		{
		"success": true,
		"message": "User retrieved from cache",
		"source": "cache",
		"payload": {
		"id": 4,
		"name": "John Doe",
		"username": "johndoe",
		"email": "john@example.com",
		"phone": null,
		"balance": 0,
		"created_at": "2026-04-16T13:55:43.135Z"
		}
		}

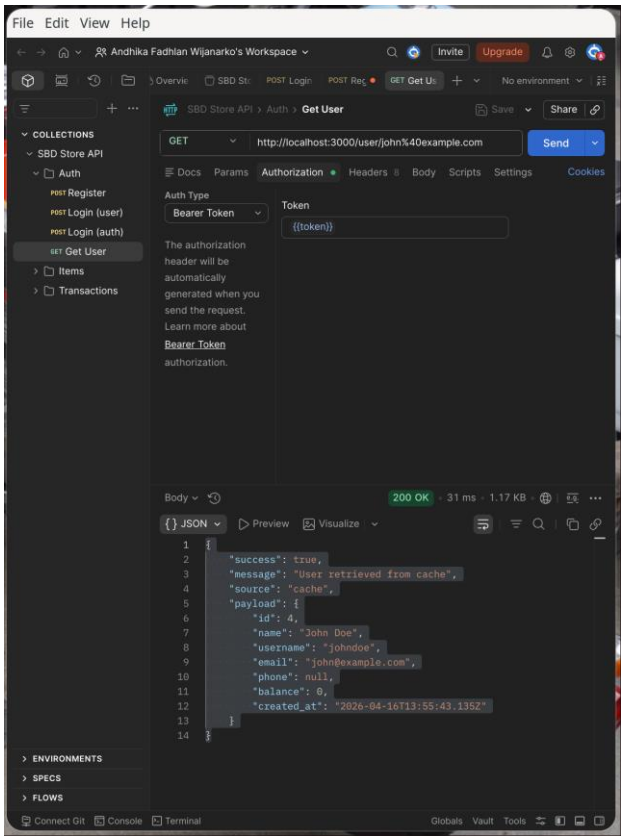
Screenshot:

static async findByEmail(email) {
const result = await db.query(
'SELECT id, name, username, email, phone, balance, created_at FROM users WHERE email = $1',
[email]
);
return result.rows[0];
}
static async findById(id) {
const result = await db.query('SELECT id, name, username, email, phone, balance, created_at FROM users WHERE id = $1', [id]);
return result.rows[0];
}
static async update(id, { name, username, email, phone, password, balance }) {
const result = await db.query(
'UPDATE users SET
name = COALESCE($1, name),
username = COALESCE($2, username),
email = COALESCE($3, email),
phone = COALESCE($4, phone),
password = COALESCE($5, password),
balance = COALESCE($6, balance)
WHERE id = $7
RETURNING id, name, username, email, phone, balance',
[name, username, email, phone, password, balance, id]
);
return result.rows[0];
}
static async getTransactionHistory(userId) {
const result = await db.query(
'SELECT t.id, i.name AS item_name, t.quantity, t.total, t.status, t.created_at
FROM transactions t
JOIN items i ON t.item_id = i.id
WHERE t.user_id = $1
ORDER BY t.created_at DESC',
models src 29/1

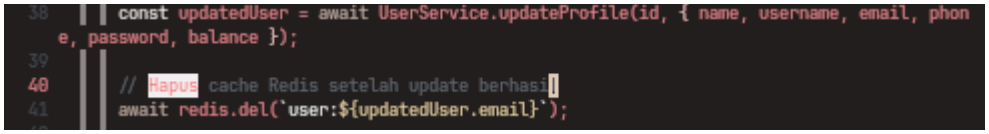
{ userId: user.id, email: user.email },
process.env.JWT_SECRET,
{ expiresIn: '24h' }
);
return { token, user: { id: user.id, name: user.name, username: user.username, email: user.email, phone: user.phone, balance: user.balance } };
}
static async updateProfile(id, updateData) {
// If password is being updated, hash it
if (updateData.password) {
const salt = await bcrypt.genSalt(10);
updateData.password = await bcrypt.hash(updateData.password, salt);
}
const updatedUser = await User.update(id, updateData);
if (!updatedUser) {
throw new AppError('User not found', 404);
}
return updatedUser;
}
static async getTransactionHistory(userId) {
return await User.getTransactionHistory(userId);
}
static async getTotalSpent(userId) {
return await User.getTotalSpent(userId);
}
static async findByEmail(email) {
return await User.findByEmailPublic(email);
}
}
module.exports = UserService;
src 74/16
/getuser


const UserService = require('../services/user.service');
const { AppError } = require('../middleware/errorHandler');
const redis = require('../database/redis');
class UserController {
static async register(req, res, next) {
try {
const { name, username, email, phone, password } = req.body;
const user = await UserService.register({ name, username, email, phone, password });
res.status(201).json({
success: true,
message: 'User registered successfully',
payload: user,
});
} catch (error) {
next(error);
}
}
static async login(req, res, next) {
try {
const { email, password } = req.body;
const { token, user } = await UserService.login(email, password);
// Return only user data (no token) for /user/login
res.status(200).json({
success: true,
message: 'Login successful',
payload: user,
});
} catch (error) {
next(error);
}
}
static async updateProfile(req, res, next) {
try {
src 1/1

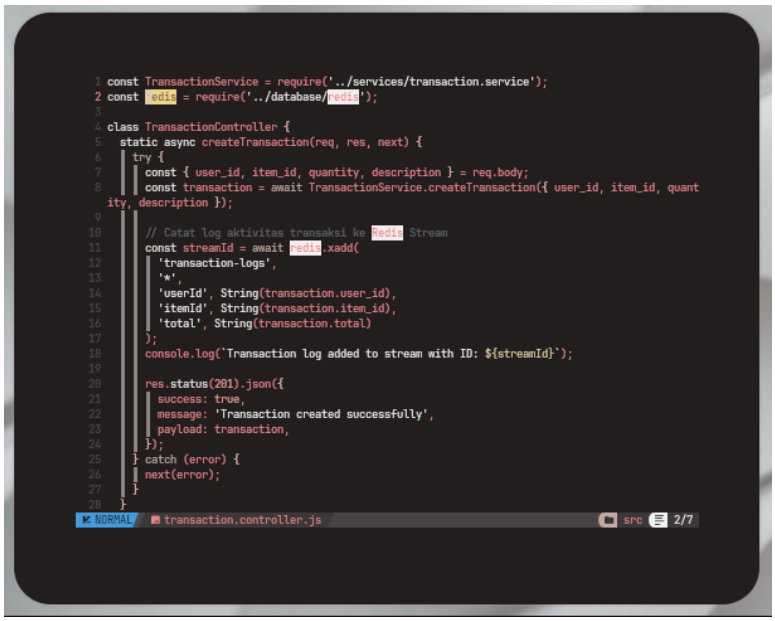
static async getUserByEmail(req, res, next) {
try {
const { email } = req.params;
const cacheKey = `user:${email}`;
// Cache-aside: cek Redis dulu
const cached = await redis.get(cacheKey);
if (cached) {
return res.status(200).json({
success: true,
message: 'User retrieved from cache',
source: 'cache',
payload: JSON.parse(cached),
});
}
// Cache miss: ambil dari database
const user = await UserService.getUserByEmail(email);
if (!user) throw new AppError('User not found', 404);
// Simpan ke Redis dengan TTL 60 detik
await redis.set(cacheKey, JSON.stringify(user), 'EX', 60);
res.status(200).json({
success: true,
message: 'User retrieved from database',
source: 'database',
payload: user,
});
} catch (error) {
next(error);
}
}
static async getTotalSpent(req, res, next) {
try {
src 46/25
/user/



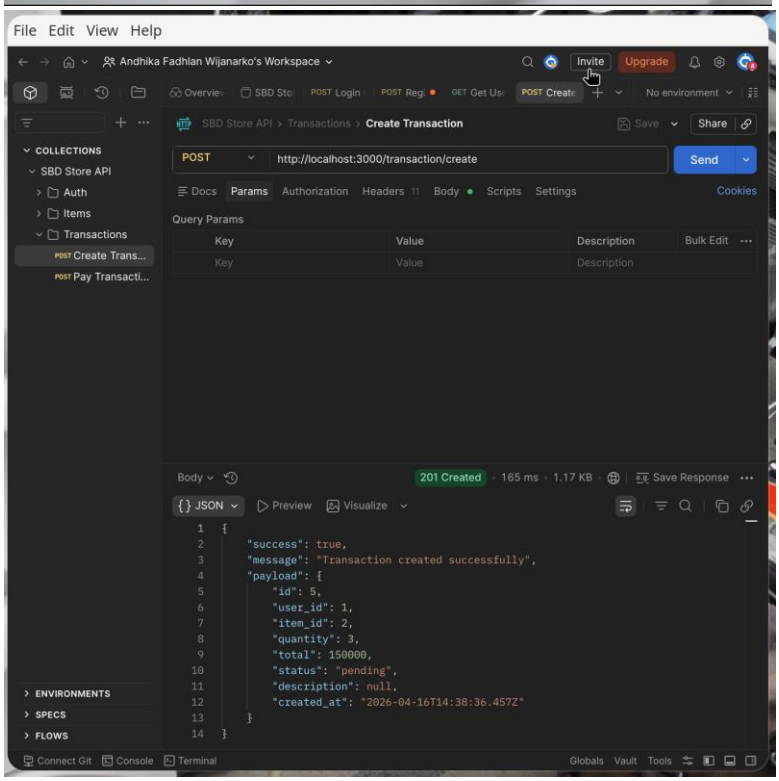
3. Cache Invalidation pada Update User
   Screenshot:



4. Logging Transaksi menggunakan Redis Streams
   Screenshot:



```
const TransactionService = require('../services/transaction.service');
const redis = require('../database/redis');

class TransactionController {
  static async createTransaction(req, res, next) {
    try {
      const { user_id, item_id, quantity, description } = req.body;
      const transaction = await TransactionService.createTransaction({ user_id, item_id, quant
ity, description });

      // Catat log aktivitas transaksi ke Redis Stream
      const streamId = await redis.xadd(
        'transaction-logs',
        '*',
        'userId', String(transaction.user_id),
        'itemId', String(transaction.item_id),
        'total', String(transaction.total)
      );
      console.log(`Transaction log added to stream with ID: ${streamId}`);

      res.status(201).json({
        success: true,
        message: 'Transaction created successfully',
        payload: transaction,
      });
    } catch (error) {
      next(error);
    }
  }
NORMAL  transaction.controller.js                                  src  2/7
```



5. Berikut perbandingan hasil pengukurannya:

	PostgreSQL (Cache Miss)	Redis (Cache Hit)
Latency curl	0.040056s (40ms)	0.004986s (5ms)
Latency server	29ms	4ms
Sumber data	PostgreSQL	Redis

Redis ~8x lebih cepat dari PostgreSQL.

Penjelasan perbedaan latensi:

- PostgreSQL (40ms), request harus melalui proses:
    1. parsing query SQL
    2. koneksi ke database
    3. eksekusi query
    4. hasil dikembalikan ke aplikasi.

    Proses ini melibatkan I/O disk dan network ke database server.

- Redis (5ms), data sudah tersimpan di memory, tidak ada query SQL, tidak ada I/O disk. Redis langsung membaca key user:john@example.com dan mengembalikan data dalam hitungan milidetik.

Cache-aside dengan Redis sangat efektif untuk data yang sering diakses (read-heavy) seperti profil user. Dengan latensi 8x lebih cepat, Redis mengurangi beban query ke PostgreSQL secara signifikan, terutama saat trafik tinggi.

Screenshot: